Team Details

a. **Team name:** perceptron
b. **Team leader name:** Aman Kumar Maurya

Sourced - product intelligence that proves where every value came from, and refuses to guess.

Brief about your solution

Sourced - enrichment that proves its sources, and refuses to guess

A distributor does not start with a datasheet. They start with a row like this:

```
MPN: 3GAA132214-ADE   MFR: ABB   DESC: MOT 3GAA132 5.5KW 4P B3
```

No attached document. A sellable record needs 80-120 attributes. Hand the fragment to an LLM and you get plausible, unsourced, occasionally wrong specifications. In industrial supply a wrong pressure rating means the wrong part arrives on a job site.

Confidently wrong is worse than absent.

- **Finds the source first.** Verifies a document names this exact part before reading a value from it. No verified source, no extraction.
- **Publishes with provenance.** Every value carries the document, page and bounding box. Click it and the PDF region is outlined.
- **Measures its own confidence.** Fitted against labels and reported as a reliability diagram, not asked of the model.
- **Explains refusals.** A typed abstention with a reason and what would resolve it - a work item, not a blank.

MEASURED - held-out split, 207 SKUs

99.6%
precision on published values

76.9%
auto-publishable, no human review

24/24
wrong-part traps refused

0.006
expected calibration error

100.0%
source located and verified

100.0%
routed to the right schema

Two verticals - 656 SKUs, 138 documents, 471 listings.
Reproducible: python -m sourced.eval.report

1. How does your solution enrich minimal product information?

Briefly explain how your solution transforms limited inputs (e.g., Part Number, Brand, Short Description) into rich product intelligence.

2. How does your solution ensure accuracy and trust in the generated product data?

Describe your validation strategy. This may include:
(Confidence scoring, Multi-source verification, Human review, Rule-based validation, AI validation, Other approaches)

3. What makes your solution scalable for enterprise product catalogs?

Describe how your solution would handle: Large product catalogs, New manufacturers, Different document formats, Continuous product updates

Answered in full on the next three slides. In one line each:

#	In one line	Evidence
1	Seven stages turn MPN + brand + fragment into a commerce record. The source is located and verified before anything is read.	**100.0% located**
2	Six independent layers, each measured. Deterministic gates take an LLM's output from 33.3% correct to 100.0%.	**99.6% precision**
3	A category is a YAML file, not a code change. The model is called once per SKU, for unresolved attributes only.	**4,416 SKUs/min**

Every figure in this deck is measured on a held-out split and reproducible with a single command. Sources: docs/RESULTS.md

1. How does your solution enrich minimal product information?

Stage	What it does
0 Source discovery	Normalise the MPN, resolve the manufacturer alias, retrieve candidates (BM25 + dense, fused by RRF), then verify the document names THIS part. No verified source -> no_source_located, with a reason.
1 Classification	Embed taxonomy leaves offline, retrieve top-k, constrain the choice to real codes, hard-validate the code exists. An invented code is structurally impossible.
2 Candidates	Three tiers, none short-circuits: rules (lexicon + dimensional grammar), tables (row chunks with per-cell bounding boxes), LLM (one structured call for whatever is left).
3 Adjudication	Rank by evidence quality, then source authority, then independent agreement. Two manufacturer datasheets disagreeing is a conflict, not a vote.
4-5 Validate + calibrate	Per-attribute, cross-attribute and cross-SKU checks; then a publish threshold set by the attribute's criticality tier.
6-7 Commerce + ops	Deterministic title, description with claim traceability, facets; idempotent upsert with source-change detection.

Three tiers propose; none of them decides alone

Rules read the distributor's own row. Tables read the document, one row at a time, keeping each cell's bounding box. The model is asked only for what the other two could not resolve - one structured call per SKU, never one per attribute. Adjudication then ranks the proposals by evidence quality before source authority before agreement, so a regex that happens to match cannot beat a table cell.

Measured: 100.0% source-location rate | 100.0% of attributes resolved with no LLM call | 85.7% recall

The brief's own example, decoded

```
1/2IN X 3/4IN BRS 90 ELL
FIP 150#
```

```
+ nominal_size                0.5"
+ nominal_size_secondary      0.75"
+ body_material               brass
+ bend_angle                  90
+ form_factor                 elbow
+ end_connection_1            female_iron_pipe
+ pressure_class              class_150
```

Seven attributes. No document. No model. Rules alone, auditable to the characters they fired on.

74% / 90%

attribute coverage, connector / fitting

2. How does your solution ensure accuracy and trust?

Layer	What it catches	Measured
Match verification	Extraction from a SIBLING part's datasheet - the failure every naive system commits	**24/24 refused**
Span containment	A value citing text that is not in the source it names	**38.6% of proposals cut**
Value / evidence pairing	A real span paired with a fabricated value	**88.6% total rejection**
Adjudication by authority	Three listings that copied one wrong datasheet are not three confirmations	**datasheet wins**
Validation L1 / L2 / L3	Impossible value pairs; outliers against sibling SKUs	**100% of 176 caught**
Calibrated abstention	Anything below the bar for its criticality tier	**ECE 0.006**

Confidence is computed, never self-reported

Asking a model to rate its own certainty produces numbers that look meaningful and do not track correctness. Ours is a fitted function of observable signals - which tier produced the value, whether the span check passed, how many independent sources agreed - trained on a calibration split the reported numbers never touch, and published as a reliability diagram so the threshold means something.

We removed the guard rails

Same SKUs, same model, gates off:

gates OFF

33% correct

gates ON

100% correct

With the gates off it also invented values for 8 attributes those parts do not have. The cheapest check in the system - a substring test - does the most work.

Thresholds are a policy

A wrong housing colour is a cosmetic defect. A wrong current rating is a fire. Safety attributes need 0.99 confidence AND two independent sources; cosmetic ones publish at 0.85.

Precision by tier - safety 100.0% | functional 99.9% | cosmetic 97.0% (1,897 published values scored)

3. What makes your solution scalable for enterprise catalogs?

Concern	How it is handled	Measured
Large catalogs	Deterministic tiers resolve most of the schema. The model is called once per SKU, for unresolved attributes only - never per attribute.	**4,416 SKUs/min**
New manufacturers	Alias table plus fuzzy resolution with a hard floor: below it nothing resolves, rather than binding the SKU to the wrong maker.	**15% arrive with none**
New categories	An attribute set, its criticality tiers, relational rules and lookups are a YAML file. No code changes.	**100.0% routing**
Document formats	Datasheets, catalogue pages and JSON listings all become the same chunk type with the same provenance shape.	**138 docs + 471 listings**
Continuous updates	Content-hash change detection re-verifies only affected products, and only the attributes that actually moved.	**7.2% of a full re-run**

Cost scales with what is missing, not with catalogue size

Because the model only sees attributes the deterministic tiers could not resolve - and is not called at all when they suffice - a distributor with a million SKUs pays for the gaps, not the rows. In the reported run that cost was $0.00 per 1,000 SKUs.

4,416
SKUs per minute, single process

100.0%
of attributes never reached a model

7.2%
the cost of a source revision

73
taxonomy leaves; a category is a YAML file

Opportunities

a. How different is it from other existing ideas?

Most enrichment demos start from a PDF you already have, and end at a JSON blob.

Typical approach	Sourced
Starts from a datasheet you supply	**Starts from a part number and FINDS the document - then proves it is the right one**
Values with no provenance	Every value carries document, page and bounding box
The model self-reports its confidence	Confidence fitted against labels, published as a reliability diagram
Silence when it fails	A typed abstention with a reason and a resolution hint
One LLM call per attribute	One call per SKU, unresolved attributes only

b. How will it solve the problem statement?

Each of the brief's four outcomes has a metric attached, not an assertion.

Outcome	Metric	Result
Structured data generation	source location - coverage	**100% - 74%/90%**
Accuracy and consistency	precision - unit uniformity	**99.6% - 100%**
AI validation and enrichment	auto-publish - fabrication caught	**76.9% - 100%**
Scalable catalog engine	throughput - re-verification cost	**4,416/min - 7.2%**

c. USP of the proposed solution

> ***The only value it publishes is one it can point at - and it says so out loud when it cannot.***

Three things a rival deck is unlikely to have:

- **A wrong-part test as a build gate.** 24/24 refused, every time.
- **A reliability diagram.** ECE 0.006. Everyone shows confidence scores; almost nobody shows theirs are calibrated.
- **Honest negatives.** Three ablations moved nothing and we say why - including one place our own design prediction was wrong.

List of features offered by the solution

ENRICHMENT

- Source discovery from a bare part number
- Hard MPN-presence gate before extraction
- Boundary-anchored matching: a truncated MPN cannot match a longer part
- Manufacturer alias resolution with a confidence floor
- Rules tier: abbreviation lexicon + dimensional grammar
- Tables tier: row chunks with per-cell bounding boxes
- LLM tier: one structured call, unresolved attributes only
- Unit normalisation - 1/2 in, 0.5", 12.7 mm are one value
- MPN structure decoding, learned unsupervised

TRUST

- Verbatim span containment on every tier
- Value / evidence pairing check on model output
- Adjudication: evidence, then authority, then agreement
- Conflicting datasheets produce an abstention, not a vote
- Validation L1 attribute, L2 relational, L3 family coherence
- Confidence calibrated per category and criticality tier
- Reliability diagram and abstention curve
- Six typed abstentions, each with a resolution hint

COMMERCE + ENGINE

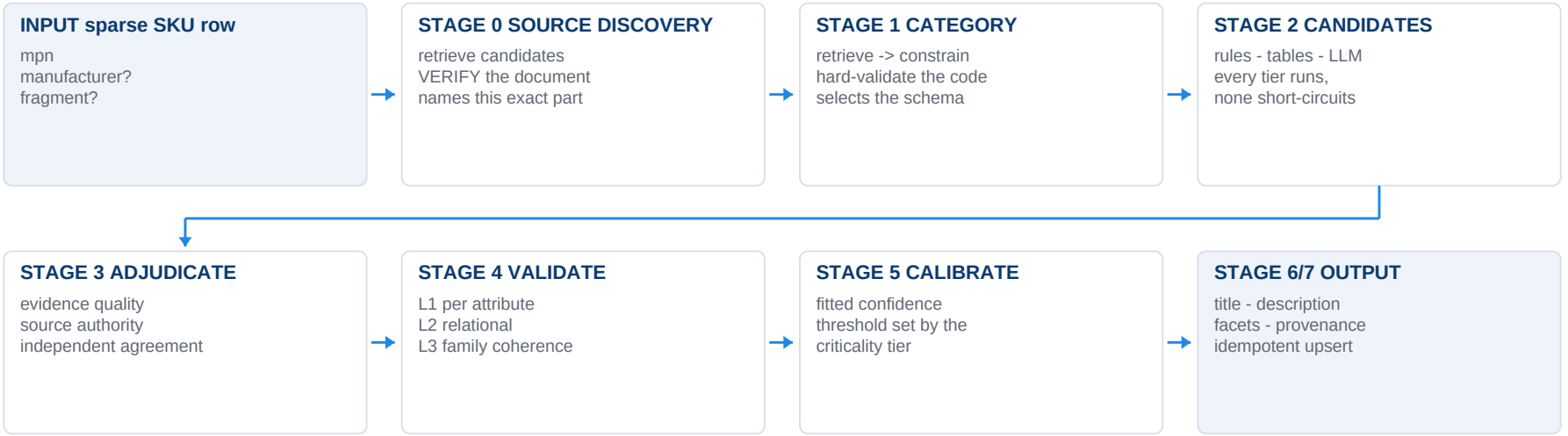
- Deterministic title - cannot hallucinate
- Description with claim-to-attribute traceability
- Numeric licence gate on generated copy
- Facets, completeness score, blocking-for-publish list
- Idempotent upsert on (manufacturer, normalised MPN)
- Source-change detection, selective re-verification
- Web UI: confidence, tier badge, reason, PDF region
- Postgres or SQLite; docker compose up
- 57 gate tests, ablations, adversarial suite, scripted demo

What ties the list together

Every feature above is either a way of finding evidence, a way of checking it, or a way of showing it. Nothing in the system asks the reader to take a value on trust: a published attribute can be clicked through to the page and region it was read from, and a missing one names the reason it is missing and what would resolve it.

Full feature list and evidence: docs/RESULTS.md | Run it: docker compose up -> http://localhost:8000

Process flow diagram or Use-case diagram



TERMINAL STATES - every one carries a reason AND what would resolve it

`no_source_located`	the MPN is in no document
`sources_conflict`	two datasheets disagree
`failed_validation`	a deterministic check rejected it
`below_threshold`	under the bar for its criticality tier

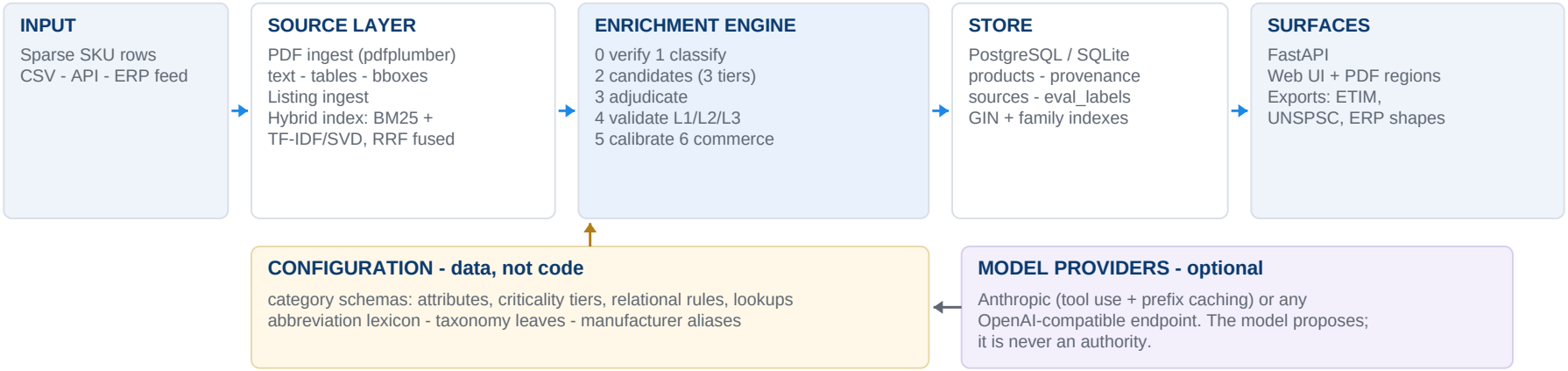
A blank field is not actionable; a named blocker is a work item. No stage short-circuits on a threshold - every tier's candidates compete, and the losers are kept so the record can explain why a value won.

Wireframes/Mock diagrams of the proposed solution (optional)

Three panes, one screen. Clicking any published value outlines the exact region it was read from.

Screenshots of the running application are on the next slide. Live at http://localhost:8000 after docker compose up.

Architecture diagram of the proposed solution



Two rules the architecture enforces structurally

- **Nothing reaches the record without evidence that survives a deterministic check.** The model is a proposer, never an authority.
- **Labels live in eval_labels with no code path from the pipeline.** A gate test walks the AST to prove it.

Technologies used in the solution

Concern	Choice	Why
Language	Python 3.12	Ecosystem for every component below
PDF text, tables, coordinates	pdfplumber	Text, tables and bounding boxes from one library, no GPU
Lexical retrieval	rank_bm25	Part numbers are exact-match tokens
Dense retrieval	scikit-learn TF-IDF + SVD	Semantic half without a 2.5 GB torch dependency
Fusion	Reciprocal Rank Fusion	Parameter-light, no score normalisation
Units	pint	Correct unit algebra and dimensionality checks
Fuzzy matching	rapidfuzz	Manufacturer aliases, attribute-key matching
Calibration	scikit-learn logistic regression	Interpretable coefficients, small-data friendly
Canonical model	pydantic v2	Type-safe schemas and provenance records
Store	PostgreSQL + JSONB (SQLite dev)	One database: attributes JSONB, provenance relational
API and UI	FastAPI + static SPA	Typed, async; renders PDF regions server-side
LLM	Anthropic or OpenAI-compatible	One provider interface; measured on Mistral-Nemo-Instruct-2407
Packaging and tests	Docker Compose, pytest	Verified from a clean volume; 57 gate tests

Deliberately excluded

Graph database (provenance is a foreign key, not a traversal) - object store (local disk) - separate search engine (Postgres suffices) - multi-agent critic loop (a model checking a model is weaker evidence than a deterministic check) - web-search enrichment (an unranked source undermines the provenance claim).

Estimated implementation cost (optional)

$0.00 per 1,000 SKUs, measured

In the reported run the deterministic tiers resolved every attribute, so no tokens were spent.

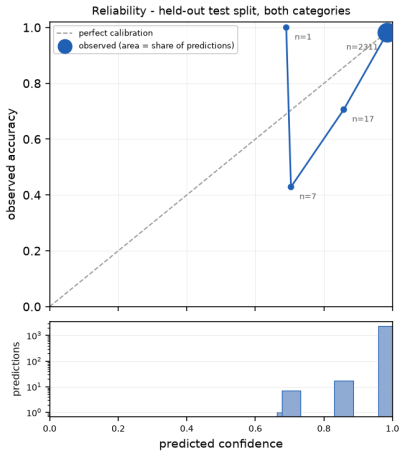
Item	Cost	Basis
Deterministic path (rules + tables)	$0	CPU only - 4,416 SKUs/min, single process
LLM tier, when it fires	~1,700 prompt tokens/SKU	Unresolved attributes only, one call per SKU
at open-weight rates (measured)	~$0.30 / 1,000 SKUs	Mistral-Nemo via an OpenAI-compatible endpoint
at frontier rates	~$5-8 / 1,000 SKUs	If a stronger model is required
Infrastructure	$50-80 / month	One small VM plus managed Postgres at this scale
Re-processing a source revision	7.2% of a full re-run	Only affected products, only changed attributes

The economics of the design

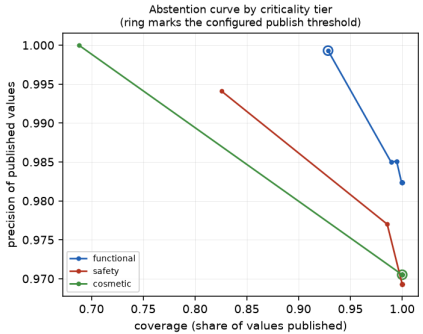
Because the model is called once per SKU for unresolved attributes only - and not at all when the deterministic tiers suffice - cost scales with what is MISSING, not with catalogue size. A distributor with a million SKUs pays for the gaps, not the rows. 100.0% of attributes in the reported run never reached a model at all.

Snapshots of the MVP

Calibration artefacts below are generated by the evaluation run. Application screenshots: replace the two panels on the right.



Reliability diagram - held-out split



Abstention curve by criticality tier

SCREENSHOT 1 - the record

published / review / abstained together, with confidence bars, tier badges and an abstention reason

SCREENSHOT 2 - the provenance panel

the PDF page with the cited cell outlined - the shot that carries the whole pitch

ECE 0.006 - predicted confidence tracks observed accuracy. Everyone displays confidence scores; almost nobody demonstrates theirs are calibrated.

Additional Details/Future Development (if any)

WHAT IS MEASURED TODAY

Corpus	656 SKUs, 2 verticals, 138 documents, 471 listings
Held-out test split	207 SKUs, touched once
Gate tests	57, including the wrong-part build gate
Deployment	docker compose up verified from a clean volume, on Postgres

STATED HONESTLY

- **The corpus is generated, not pulled from a distributor API.** Digi-Key and Mouser keys need interactive registration. Labels therefore carry no noise by construction, and real documents are messier. The Digi-Key ingestion path is implemented and needs only a key.
- **The LLM tier is measured on samples against a 12B open-weight model,** not the frontier model the design assumes.
- **Two of 73 taxonomy leaves have populated attribute schemas.**

NEXT

1. **Real corpus**
   Swap the generator for the Digi-Key path, re-measure, and report label noise from a hand audit
2. **More categories**
   Each is a YAML file; the pipeline does not change
3. **OCR path**
   For scanned datasheets, the one document class not handled
4. **Steward workflow**
   Abstentions are already work items with resolution hints - queue and assign them
5. **Standards projections**
   ETIM and UNSPSC exports off the canonical model
6. **Word-level licence gate**
   Today's gate is numeric; a fabricated phrase with no digits would pass

Provide links to your:

1. GitHub Public Repository — github.com/<your-handle>/sourced
2. Demo Video Link (3 Minutes) — <paste the 3-minute video link>
3. Working Prototype Link — docker compose up -> http://localhost:8000

RUN IT IN FOUR COMMANDS

```
$ pip install -e ".[dev]"
$ python -m sourced.corpus.build
$ python -m sourced.eval.report --persist --fresh
$ uvicorn sourced.api.routes:app --port 8000
```

Or: docker compose up - python -m sourced.demo narrates the system in six acts, with an offline replay fallback.

IN THE REPOSITORY

docs/RESULTS.md	every measured number, ablations, limitations
docs/SUBMISSION.md	this deck, in full
docs/DEMO_SCRIPT.md	the 3-minute script
docs/00-08	problem, architecture, data model, pipeline spec, evaluation, sources, build plan, risks, decisions

Sourced turns a part number into a sellable record with the page and region every value came from - and returns an explained refusal instead of a plausible guess.

unilog | H2S HACK2SKILL

UniHack

AI-Powered Product Intelligence for
Industrial Commerce

Thank You